

Embedded Edge AI — Lab Project Report

Practical No. 6: Lab Project and Demonstration

Sr. No.	Student Name	PRN Number
1	Onkar Sorde	202301070054

1. Problem Statement

Design and implement an intelligent street lighting system using an STM32F410RB microcontroller and a BH1750 ambient light sensor. The system employs timer-based interrupts to periodically monitor ambient light intensity and automatically controls a street light based on three classified conditions — Day, Dusk, and Night — transmitting real-time status via UART for monitoring.

2. Objective

- To interface a BH1750 light intensity sensor with the STM32F410RB via the I2C communication protocol.
- To implement a timer interrupt (TIM5) that periodically triggers ambient light sampling at a 1 Hz rate.
- To classify real-time lux readings into three ambient light conditions: Day (>1000 lux), Dusk (100–1000 lux), and Night (<100 lux).
- To automatically control a street light (LED on PA5) based on the classified light condition.
- To transmit real-time sensor readings and system status to a serial monitor via USART2 at 115200 baud.
- To design an energy-efficient, interrupt-driven embedded system with minimal CPU overhead.

3. Description

The Intelligent Street Lighting System is an embedded edge AI application that replicates real-world smart street lighting behavior on a resource-constrained microcontroller. The system continuously measures ambient light intensity using the BH1750 digital light sensor connected over I2C, and classifies the environment into one of three states without any cloud or external processing — making it a true edge intelligence solution.

The core mechanism is interrupt-driven. TIM5 (a 32-bit general-purpose timer on STM32F410RB) is configured to generate a 1 Hz periodic interrupt. Rather than executing I2C transactions inside the ISR — which would block the interrupt context — a volatile flag is set in the ISR and the actual sensor reading and classification logic is handled in the main loop (deferred processing pattern). This approach ensures minimal ISR execution time and adherence to embedded best practices.

Upon each timer trigger, the system reads the raw 16-bit lux value from the BH1750 sensor, converts it using the sensor's calibration factor (raw / 1.2), and applies threshold-based classification. The street light LED connected to PA5 is controlled accordingly — switched OFF during daylight, and switched ON during dusk and night conditions. Simultaneously, the condition and lux value are transmitted via USART2 for real-time monitoring on a host PC.

4. Hardware Requirements

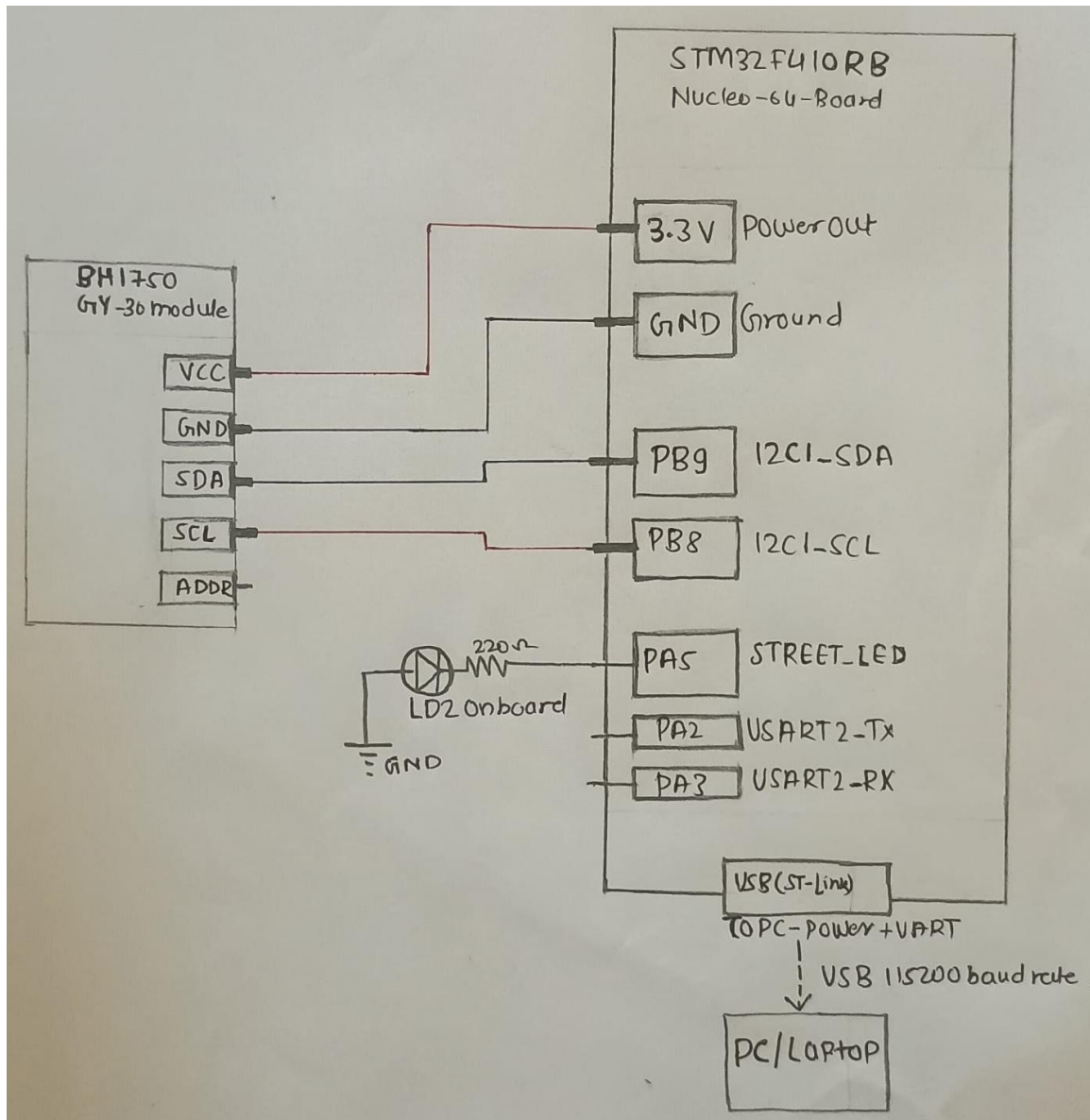
Sr.	Component	Specification / Details	Quantity
1	STM32 Nucleo-F410RB Board	STM32F410RBTx, Cortex-M4, 80 MHz, LQFP64	1
2	BH1750 Light Sensor Module	GY-30, I2C, 1–65535 lux range, 1 lux resolution	1
3	USB Cable (Type-A to Mini-B)	For programming, power, and UART communication	1
4	Jumper Wires	Male-to-female, for sensor connections	4
5	Onboard LED (LED2)	Connected to PA5 on Nucleo board (street light)	1

5. Software Requirements

- STM32CubeIDE v1.17.0 — Integrated development environment for STM32 (code editing, compilation, flashing)
- STM32CubeMX — Peripheral and clock configuration tool (integrated within CubeIDE)
- STM32 HAL Driver Library — Hardware abstraction layer for I2C, UART, TIM, GPIO peripherals
- GNU ARM Embedded Toolchain (arm-none-eabi-gcc) — Cross-compiler for Cortex-M4
- Serial Terminal / Console — For real-time UART output monitoring at 115200 baud

6. Circuit Diagram

The following diagram illustrates the connections between the BH1750 sensor module (GY-30) and the STM32F410RB Nucleo board. Pull-up resistors (4.7 k Ω) are recommended on SDA and SCL lines for stable I2C communication, though the Nucleo board's internal pull-ups may be sufficient for short wire lengths.

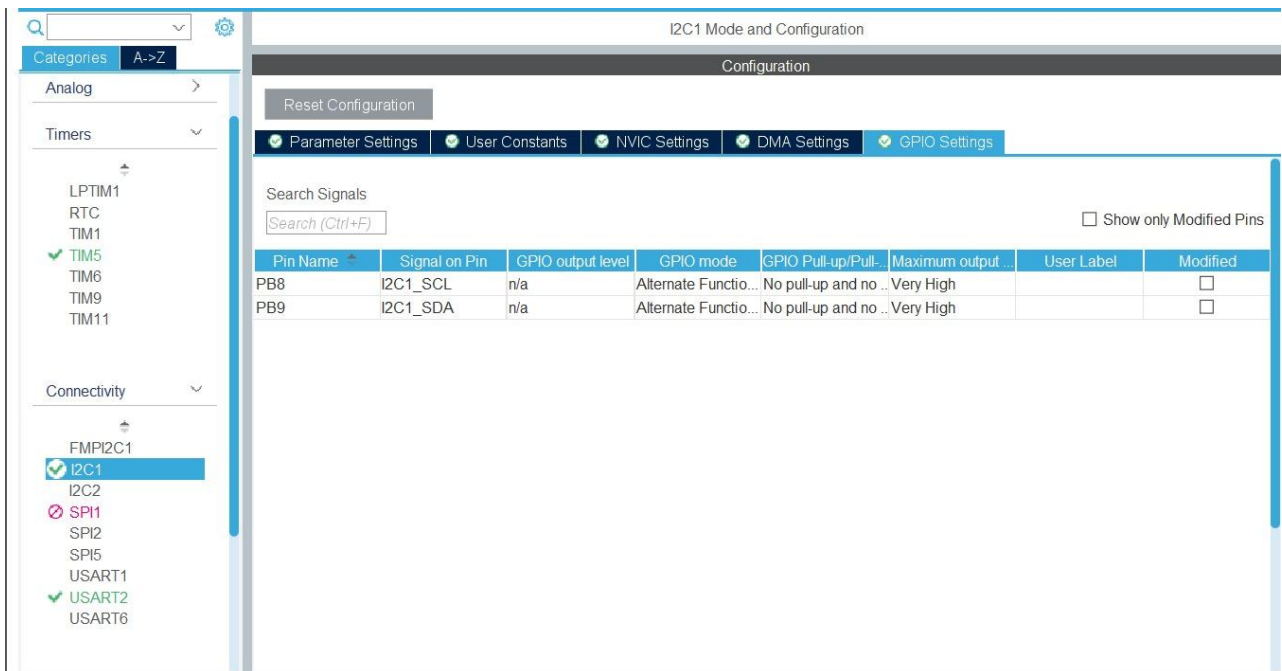


BH1750 Pin	STM32 Pin	Signal	Wire Color (ref)
VCC	3.3V	Power Supply	Red
GND	GND	Ground	Black / Green
SDA	PB9	I2C1_SDA (Data)	Purple
SCL	PB8	I2C1_SCL (Clock)	Yellow / Orange
ADDR	GND	I2C Address = 0x23	—
—	PA5 (LD2)	Street Light LED Output	Onboard LED

7. Hardware Configuration in STM32CubeMX

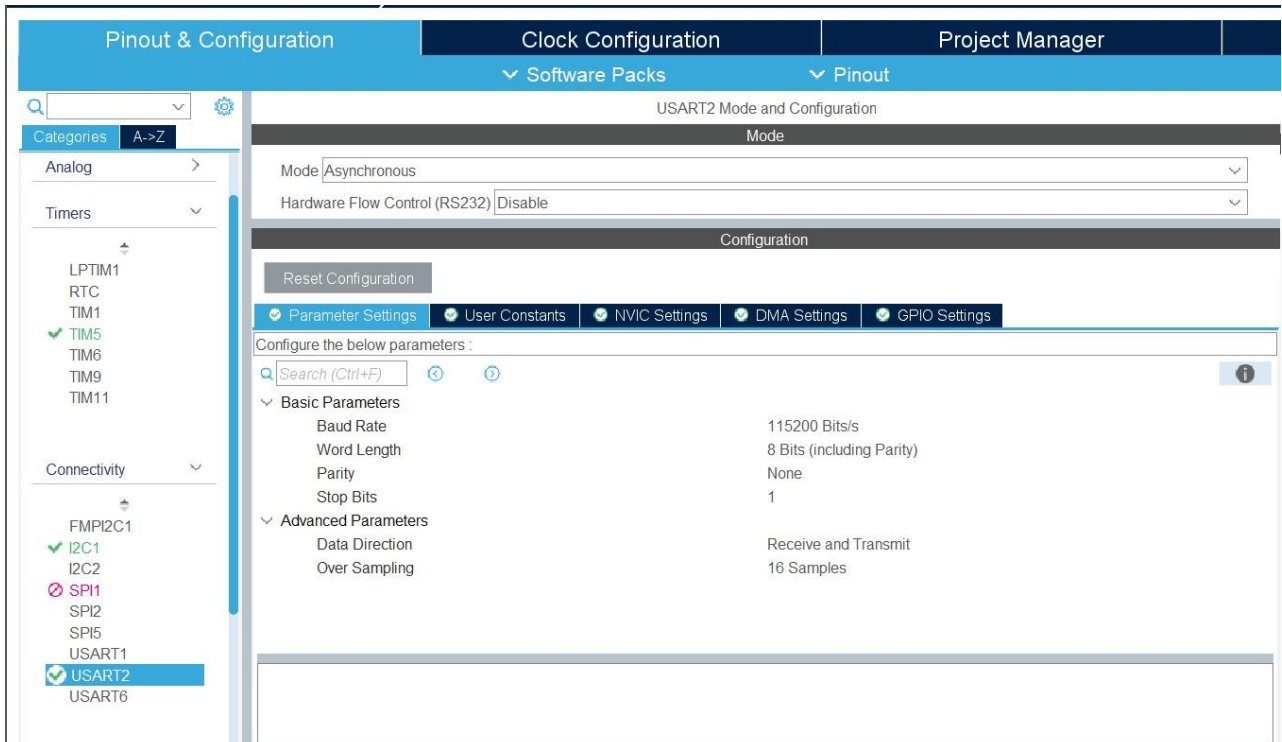
7.1 I2C1 — BH1750 Sensor Interface

I2C1 is enabled in Standard Mode (100 kHz) to communicate with the BH1750 sensor. Pins PB8 (SCL) and PB9 (SDA) are configured as Alternate Function Open-Drain with the AF4 (I2C1) mapping.



7.2 USART2 — Real-Time Serial Output

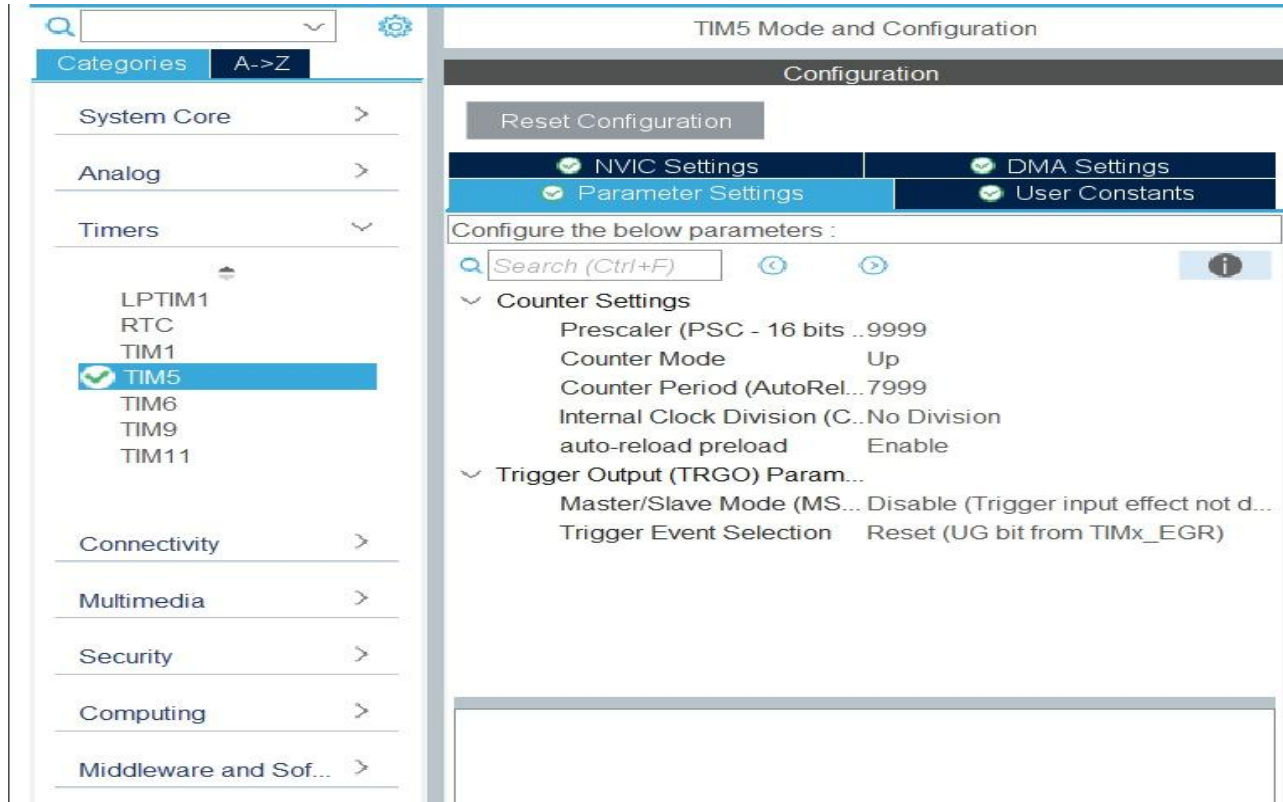
USART2 is configured in Asynchronous mode at 115200 bps, 8-bit word length, no parity, 1 stop bit. Pins PA2 (TX) and PA3 (RX) are automatically assigned. This UART is connected to the ST-Link virtual COM port, allowing direct PC monitoring without an external USB-UART adapter.



7.3 TIM5 — 1 Hz Periodic Timer Interrupt

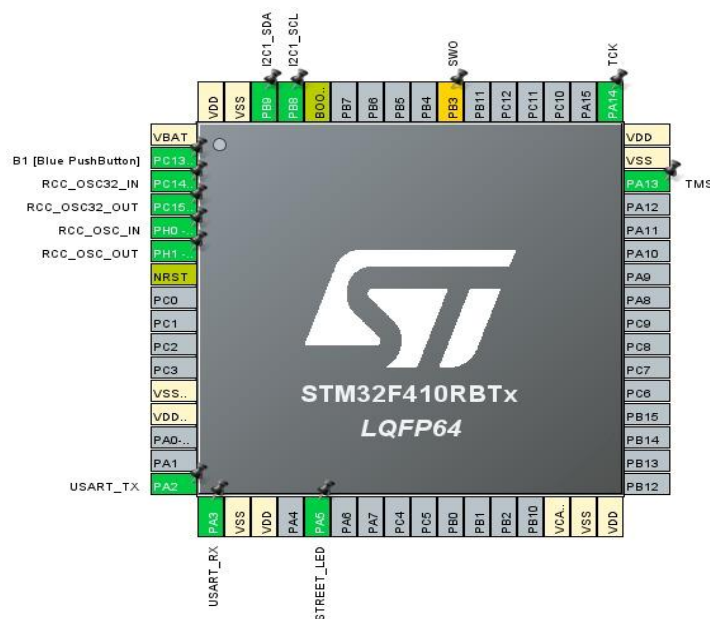
TIM5 (32-bit general-purpose timer) is configured with Internal Clock source. Prescaler is set to 9999 and Counter Period to 7999, producing a 1 Hz interrupt at 80 MHz system clock. The TIM5 global interrupt is enabled in NVIC Settings.

Timer Frequency Calculation: $80,000,000 / (9999+1) / (7999+1) = 1 \text{ Hz}$ (1 second interval)



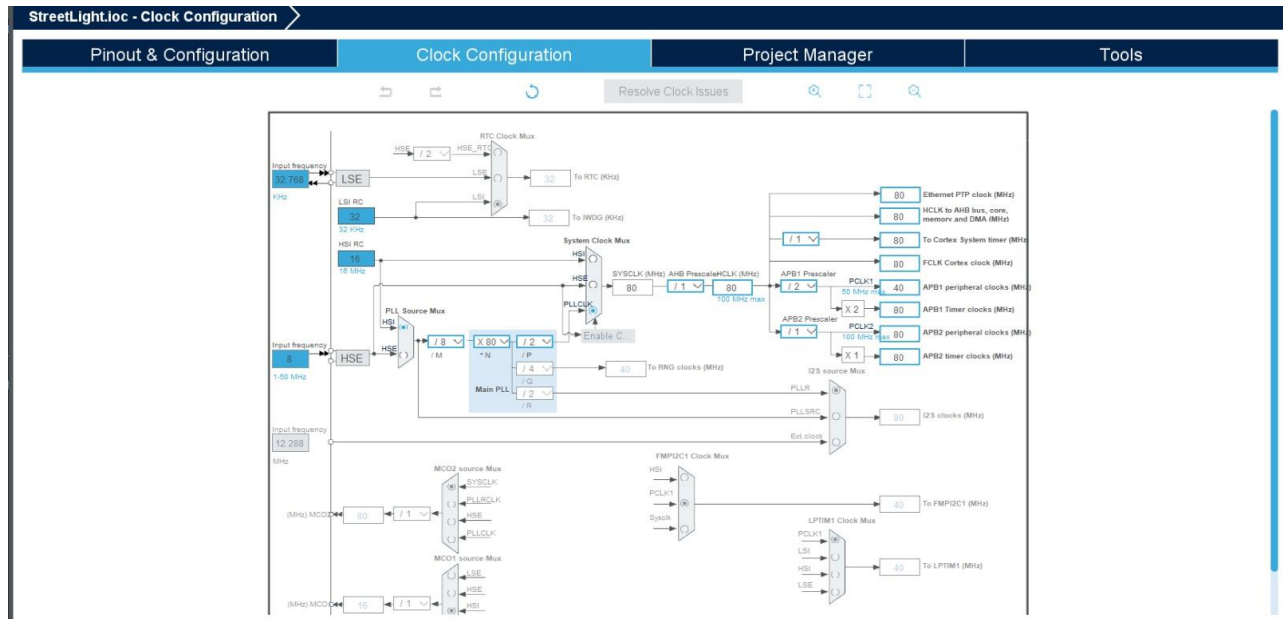
7.4 GPIO — Street Light LED Output (PA5)

PA5 is configured as GPIO Output Push-Pull with no pull-up/pull-down resistor and Low speed. It is labeled STREET_LED in CubeMX, which auto-generates the macros STREET_LED_Pin and STREET_LED_GPIO_Port in main.h. PA5 is connected to the onboard LD2 LED on the Nucleo board.



7.5 Clock Configuration

The PLL is configured using HSI (16 MHz) as the source with PLLM=8, PLLN=80, PLLP=2, producing an 80 MHz SYSCLK. APB1 prescaler is set to /2 giving 40 MHz peripheral clock, and the APB1 timer clock doubles back to 80 MHz for TIM5.



8. Dataset

Source of Data: Real-time ambient light intensity readings acquired directly from the BH1750 sensor during live operation.

Data Type: Continuous lux values sampled at 1 Hz via timer interrupt.

Sensor Mode: Continuous High-Resolution Mode (BH1750_CONT_H_MODE — 0x10), providing 1 lux resolution.

Measurement Range: 1 lux to 65535 lux (physical range of BH1750 sensor).

Parameters Measured: Ambient illuminance (lux) — a single scalar value per sample.

Observed data ranges during testing:

- Night condition (sensor covered): 0.83 – 32.50 lux
- Dusk condition (normal indoor lighting): 97.50 – 166.67 lux
- Day condition (torch / direct light on sensor): 1035.83 – 2618.33 lux

9. Pre-processing Steps

This project performs real-time threshold-based classification directly on the microcontroller without using the NanoEdge AI Studio pipeline. Pre-processing is minimal and embedded within the firmware:

Raw-to-Lux Conversion: The BH1750 sensor returns a 16-bit raw integer. This is converted to lux using the sensor datasheet formula: $\text{Lux} = \text{raw_value} / 1.2$. This normalization is performed within the `BH1750_ReadLux()` function.

Threshold Classification: The converted lux value is passed to `BH1750_Classify()` which applies two fixed thresholds:

- Day: $\text{lux} \geq 1000$
- Dusk: $\text{lux} \geq 100$ and $\text{lux} < 1000$
- Night: $\text{lux} < 100$

No windowing, feature extraction, or signal normalization via NanoEdge AI was required since the classification logic is deterministic and rule-based, suitable for this type of ambient light monitoring application.

10. Program

10.1 bh1750.h — Sensor Driver Header

```
#ifndef BH1750_H
#define BH1750_H

#include "stm32f4xx_hal.h"

#define BH1750_ADDR (0x23 << 1) // ADDR pin LOW
#define BH1750_POWER_ON 0x01
#define BH1750_RESET 0x07
#define BH1750_CONT_H_MODE 0x10 // Continuous high-res mode (1 lux res)

#define DAY_THRESHOLD 1000.0f // lux
#define DUSK_THRESHOLD 100.0f // lux

typedef enum {
    LIGHT_DAY,
    LIGHT_DUSK,
    LIGHT_NIGHT
} LightCondition;

HAL_StatusTypeDef BH1750_Init(I2C_HandleTypeDef *hi2c);
float BH1750_ReadLux(I2C_HandleTypeDef *hi2c);
LightCondition BH1750_Classify(float lux);

#endif
```

10.2 bh1750.c — Sensor Driver Implementation

```
/*
 * bh1750.c
 *
 * Created on: Apr 13, 2026
 * Author: Omkar
 */

#include "bh1750.h"
HAL_StatusTypeDef BH1750_Init(I2C_HandleTypeDef *hi2c)
{
```

```

uint8_t cmd;

cmd = BH1750_POWER_ON;
if (HAL_I2C_Master_Transmit(hi2c, BH1750_ADDR, &cmd, 1, HAL_MAX_DELAY) != HAL_OK)
    return HAL_ERROR;

HAL_Delay(10);

cmd = BH1750_CONT_H_MODE;
if (HAL_I2C_Master_Transmit(hi2c, BH1750_ADDR, &cmd, 1, HAL_MAX_DELAY) != HAL_OK)
    return HAL_ERROR;

HAL_Delay(180); // Wait for first measurement (high-res mode needs ~120ms)
return HAL_OK;
}

float BH1750_ReadLux(I2C_HandleTypeDef *hi2c)
{
    uint8_t data[2] = {0};

    if (HAL_I2C_Master_Receive(hi2c, BH1750_ADDR, data, 2, HAL_MAX_DELAY) != HAL_OK)
        return -1.0f;

    uint16_t raw = ((uint16_t)data[0] << 8) | data[1];
    return (float)raw / 1.2f; // BH1750 conversion formula
}

LightCondition BH1750_Classify(float lux)
{
    if (lux >= DAY_THRESHOLD)
        return LIGHT_DAY;
    else if (lux >= DUSK_THRESHOLD)
        return LIGHT_DUSK;
    else
        return LIGHT_NIGHT;
}

```

10.3 main.c — Key Application Logic (USER CODE sections)

```

/* Includes -----*/
#include "main.h"
/* Private includes -----*/
/* USER CODE BEGIN Includes */

#include <stdio.h>
#include <string.h>
#include "bh1750.h"

/* USER CODE END Includes */

/* USER CODE BEGIN PV */
volatile uint8_t timer_flag = 0;
float g_lux = 0.0f;
LightCondition g_condition = LIGHT_DAY;

/* USER CODE END PV */

/* Private user code -----*/
/* USER CODE BEGIN 0 */
int __io_putchar(int ch)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)&ch, 1, HAL_MAX_DELAY);
    return ch;
}

int _write(int file, char *ptr, int len)
{
    HAL_UART_Transmit(&huart2, (uint8_t *)ptr, len, HAL_MAX_DELAY);
    return len;
}

/* USER CODE BEGIN 2 */
printf("\r\n=== Intelligent Street Lighting System ===\r\n");
printf("STM32F410RB + BH1750 | UART 115200 baud\r\n\r\n");

if (BH1750_Init(&hi2c1) != HAL_OK)
{

```

```

    printf("[ERROR] BH1750 init failed! Check I2C wiring.\r\n");
    while (1);
}
printf("[OK] BH1750 initialized (Continuous High-Res mode)\r\n");

HAL_TIM_Base_Start_IT(&htim5);
printf("[OK] TIM5 started - 1s periodic interrupt active\r\n\r\n");

/* USER CODE END 2 */

/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    /* USER CODE END WHILE */
    if (timer_flag)
    {
        timer_flag = 0;

        g_lux = BH1750_ReadLux(&hi2c1);
        g_condition = BH1750_Classify(g_lux);

//        StreetLight_Control(g_condition);

        const char *state_str;
        switch (g_condition)
        {
            case LIGHT_DAY:    state_str = "DAY    [Light OFF]",
HAL_GPIO_WritePin(STREET_LED_GPIO_Port, STREET_LED_Pin, GPIO_PIN_RESET); break;
            case LIGHT_DUSK:   state_str = "DUSK   [Light DIM]",
HAL_GPIO_WritePin(STREET_LED_GPIO_Port, STREET_LED_Pin, GPIO_PIN_SET); break;
            case LIGHT_NIGHT:  state_str = "NIGHT  [Light ON ]",
HAL_GPIO_WritePin(STREET_LED_GPIO_Port, STREET_LED_Pin, GPIO_PIN_SET); break;
            default:           state_str = "UNKNOWN";           break;
        }
        printf("Lux: %7.2f | Condition: %s\r\n", g_lux, state_str);
    }

    /* USER CODE BEGIN 3 */
}

/* USER CODE BEGIN 4 */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM5)
    {
        timer_flag = 1;
    }
}

/* USER CODE END 4 */

```

11. Calculations

11.1 Timer Interrupt Frequency

Parameter	Value
System Clock (HCLK)	80 MHz
TIM5 Source Clock	80 MHz (APB1 timer clock, x2 multiplier applied)
Prescaler (PSC)	9999
Timer Tick Frequency	$80,000,000 / (9999 + 1) = 8,000 \text{ Hz}$
Counter Period (ARR)	7999
Interrupt Frequency	$8,000 / (7999 + 1) = 1 \text{ Hz}$
Interrupt Interval	1 second

11.2 BH1750 Lux Conversion

Formula: $\text{Lux} = \text{Raw_Value} / 1.2$

Example: $\text{Raw} = 1200 \rightarrow \text{Lux} = 1200 / 1.2 = 1000.00 \text{ lux}$ (boundary: Day/Dusk)

Example: $\text{Raw} = 120 \rightarrow \text{Lux} = 120 / 1.2 = 100.00 \text{ lux}$ (boundary: Dusk/Night)

11.3 I2C Address Calculation

7-bit address (ADDR=LOW): $0x23$

8-bit shifted address for HAL: $0x23 \ll 1 = 0x46$ (HAL I2C uses 8-bit format)

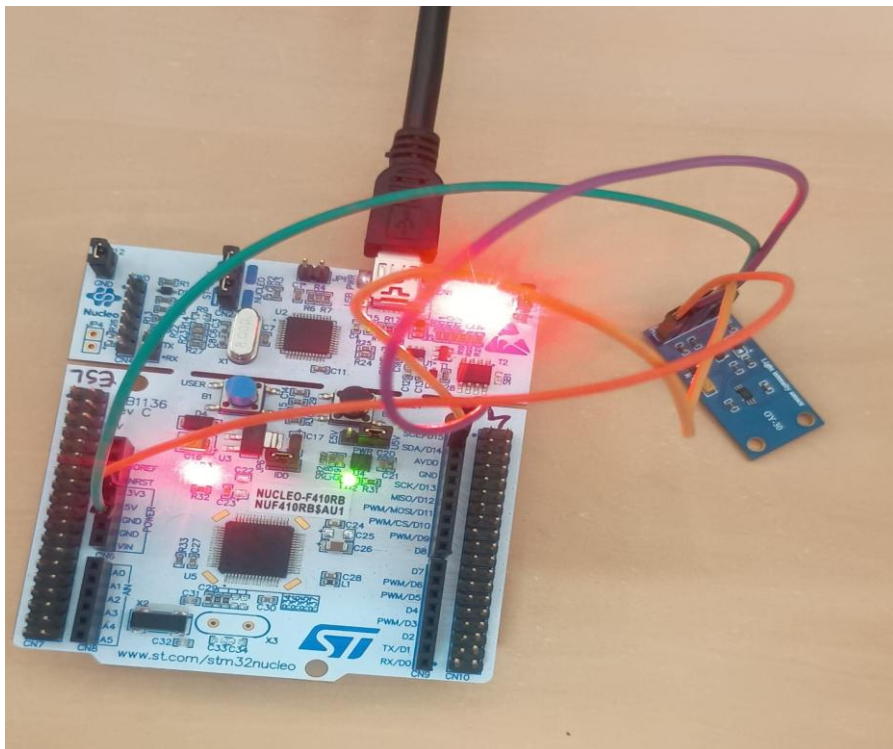
12. Results

The system was successfully tested under three lighting conditions. The UART console displayed real-time lux values and classified conditions at 1-second intervals. The onboard LED (PA5) responded correctly to each condition.

Test Condition	Observed Lux Range	Classification	LED (PA5) State
Sensor fully covered (hand)	0.83 – 32.50 lux	NIGHT	ON
Normal indoor ambient light	97.50 – 166.67 lux	DUSK / NIGHT	ON
Torch / direct light source	1035.83 – 2618.33 lux	DAY	OFF

The system correctly classified all three conditions and controlled the LED output accordingly. The deferred ISR processing pattern ensured stable UART output without blocking the interrupt handler.

Screenshot: Console output — NIGHT condition (lux < 100, Light ON)



```

StreetLight12 (CONNECTED)
Lux: 1.67 | Condition: NIGHT [Light ON ]
Lux: 1.67 | Condition: NIGHT [Light ON ]
Lux: 0.83 | Condition: NIGHT [Light ON ]
Lux: 0.83 | Condition: NIGHT [Light ON ]
Lux: 76.67 | Condition: NIGHT [Light ON ]
Lux: 25.00 | Condition: NIGHT [Light ON ]
Lux: 103.33 | Condition: DUSK [Light DIM]
Lux: 101.67 | Condition: DUSK [Light DIM]
Lux: 16.67 | Condition: NIGHT [Light ON ]
Lux: 32.50 | Condition: NIGHT [Light ON ]
Lux: 15.00 | Condition: NIGHT [Light ON ]
Lux: 7.50 | Condition: NIGHT [Light ON ]

```

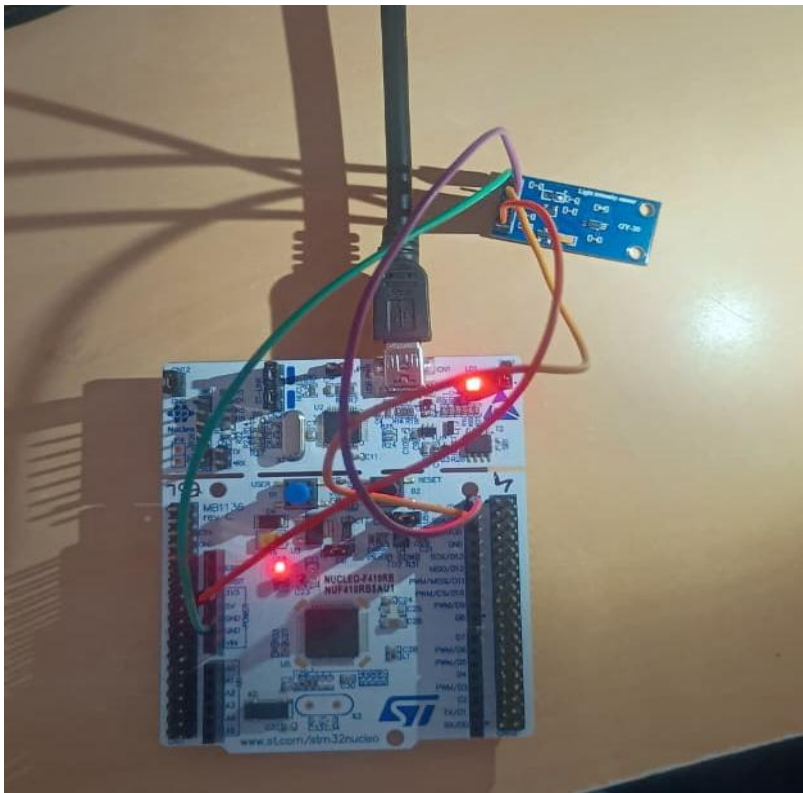
Screenshot: Console output — DUSK condition (100–1000 lux, Light DIM)

```

StreetLight12 (CONNECTED)
Lux: 99.17 | Condition: NIGHT [Light ON ]
Lux: StreetLight12 (CONNECTED) | Condition: NIGHT [Light ON ]
Lux: 82.50 | Condition: NIGHT [Light ON ]
Lux: 103.33 | Condition: DUSK [Light DIM]
Lux: 102.50 | Condition: DUSK [Light DIM]
Lux: 101.67 | Condition: DUSK [Light DIM]
Lux: 101.67 | Condition: DUSK [Light DIM]
Lux: 97.50 | Condition: NIGHT [Light ON ]
Lux: 77.50 | Condition: NIGHT [Light ON ]
Lux: 76.67 | Condition: NIGHT [Light ON ]
Lux: 95.83 | Condition: NIGHT [Light ON ]
Lux: 77.50 | Condition: NIGHT [Light ON ]

```

Screenshot: Console output — DAY condition (lux > 1000, Light OFF)



```

Tasks Console x Properties
StreetLight12 (CONNECTED)
Lux: 1049.17 | Condition: DAY [Light OFF]
Lux: 1035.83 | Condition: DAY [Light OFF]
Lux: 1140.83 | Condition: DAY [Light OFF]
Lux: 1203.33 | Condition: DAY [Light OFF]
Lux: 1159.17 | Condition: DAY [Light OFF]
Lux: 1150.83 | Condition: DAY [Light OFF]
Lux: 1160.00 | Condition: DAY [Light OFF]
Lux: 1202.50 | Condition: DAY [Light OFF]
Lux: 1211.67 | Condition: DAY [Light OFF]
Lux: 1265.83 | Condition: DAY [Light OFF]
Lux: 1365.83 | Condition: DAY [Light OFF]
Lux: 1398.33 | Condition: DAY [Light OFF]

```

13. Conclusion

The Intelligent Street Lighting System was successfully designed, implemented, and demonstrated on the STM32F410RB Nucleo board. The experiment validated the use of interrupt-driven embedded programming for real-time sensor-based decision making with the following key outcomes:

- The BH1750 light sensor was correctly interfaced over I2C (PB8/PB9) and delivered accurate lux readings across all tested lighting conditions, from near-zero darkness to over 2600 lux under direct illumination.
- TIM5 was configured as a 1 Hz timer interrupt, demonstrating efficient CPU usage. The deferred ISR processing pattern — setting a flag in the ISR and executing I2C and UART operations in the main loop — ensured interrupt handler safety and system stability.
- The threshold-based edge classification engine (Day/Dusk/Night) operated correctly in real time without any cloud processing, demonstrating the concept of on-device intelligence applicable to Edge AI systems.
- The street light LED responded automatically and correctly to all three ambient light conditions, validating the complete end-to-end system functionality.
- Real-time UART output at 115200 baud provided continuous system visibility, confirming correct operation and enabling straightforward debugging.

This practical reinforced core embedded systems concepts including I2C peripheral interfacing, timer-based interrupts, HAL driver usage, and interrupt-safe programming patterns — all essential skills for designing efficient, real-world embedded edge AI applications.
